Temporal Fusion Transforner (TFT)

Introduction to Attention Networks, Transformers, and Temporal Fusion Transformers

IEOR 4578: Forecasting: *A Real-World Application*

Syed Waseem Haider, Ph.D.

Dr.Waseem.Haider@gmail.com

April 15, 2026

Introduction

- Usually Deep Learning based forecasting techniques are black-box without giving much insights in to Whys? And Hows?
- Temporal Fusion Transformer (TFT) combines high-performance multi-horizon forecasting with interpretable insights into the time dynamics.
- TFT takes advantage of recurrent layers for local or short-term processing, and interpretable self-attention layers for long-term relationships.
- It uses specialized components to select relevant features, and a series of gating layers to suppress unnecessary compoentnts increasing the applicability in various scenarios.
- TFT uses static features, time-varying features with historic only data, and time-varying features with both historic and future data.

Introducing TFT

- Interpretability is another important characteristic of TFT that it can provide information on temporal dynamics in the data by interpreting attention patterns.
- TFT allows to analyze global temporal relationships and the emerging global behaviors or patterns such as seasonality, lagging relationships, etc.
- TFT uses a separate encoder-decoder attention for static features at each time step, and combine that with the self-attention of temporal features.
- The work presented here is based on "Temporal Fusion Transformers for Interepretable Multi-horizon Times Series Forecasting," by Bryan Lim et. al.
- Lastly, code for the implementation of TFT on the Electricity data set is available here.

Inputs and Output Space of TFT

- TFT accounts for the following types of inputs:
  - Static inputs such as demographics, geographical locations, number of trucks, brand loyalty, or other types of nominal or categorical covariates.
  - Temporal or time-based features with only historical data.
  - Temporal features with known future values such as holidays, day of the week, or other known events with data.

Inputs and Output Space of TFT

- There could be $\mathcal{I}$ entities such as products in the porfolio, or electricity customers.
- Each entity $i \epsilon \mathcal{I}$ has a set of input static attributes associated with it such as penetration, incrementality, distribution, velocity, geogrpahy, weather, etc. This set of static covariates is written as $s_i \epsilon \mathbb{R}^{m_s}$.
- Each entity i also has a number of temporal or time-varying input features associated with it such production rate, dispatch rate, inventory levels, Consumer Price Index (CPI), etc. The vector of observations at the time step t are written $\mathcal{X}_{i,t} \epsilon \mathbb{R}^{m_{\mathcal{X}}}$.
- Finally, the scalar output or the target variable for the entity i at time t is $y_{i,t} \epsilon \mathbb{R}$. This could be sales of product i, or the electricity consumption of the customer i.

Inputs and Output Space of TFT

- The temporal inputs are further divided into those $z_{i,t} \epsilon \mathbb{R}^{m_z}$ only observed up to the time step t, and the others $x_{i,t} \epsilon \mathbb{R}^{m_x}$ for which the future values could be pre-determined.
- At t time step, $\hat{y}_i$ is the forecasted outputs from $t+1$ to $t+\tau$ time steps for the entity i.
- The inputs include the historical outputs, $y_{i,t-k:t}$, historical temporal features, $z_{i,t-k:t}$, and temporal historical and future features, $x_{i,t-k:t+\tau}$ with the look-back window k.
- TFT instead of a forecasting a single output time series, provides a forecasted time series for each quantile. Which is very useful to understand the ranges within which a forecast will lie at each time step into future.
- A quantile future time series forecast for the entity i up to τ is $\hat{y}_i(q,t,\tau) \; = \; f_q(\tau, y_{i,t-k:t}, z_{i,t-k:t}, x_{i,t-k:t+\tau}, s_i)$.

TFT Architecture

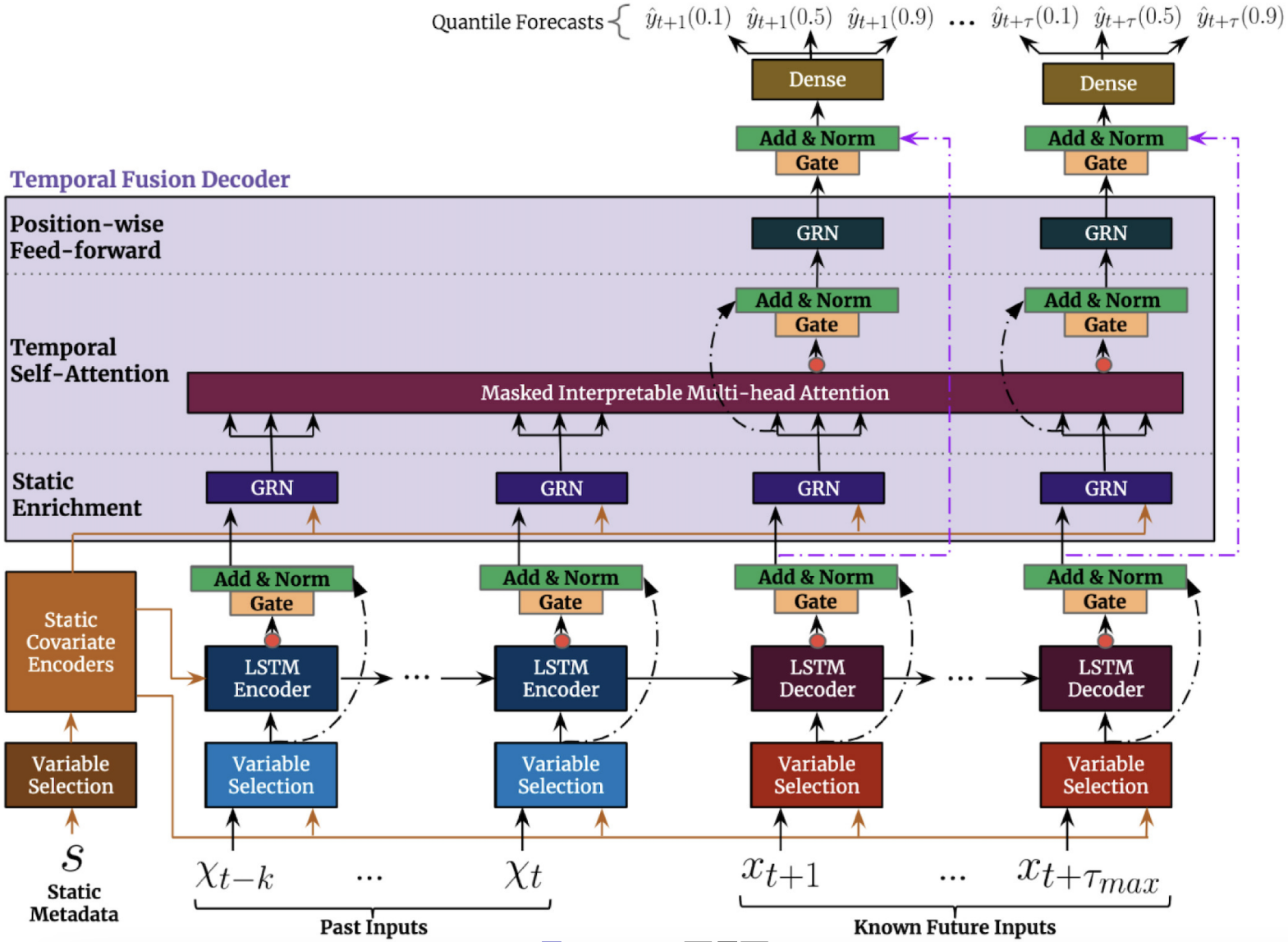

Figure: TFT

GRN Architecture

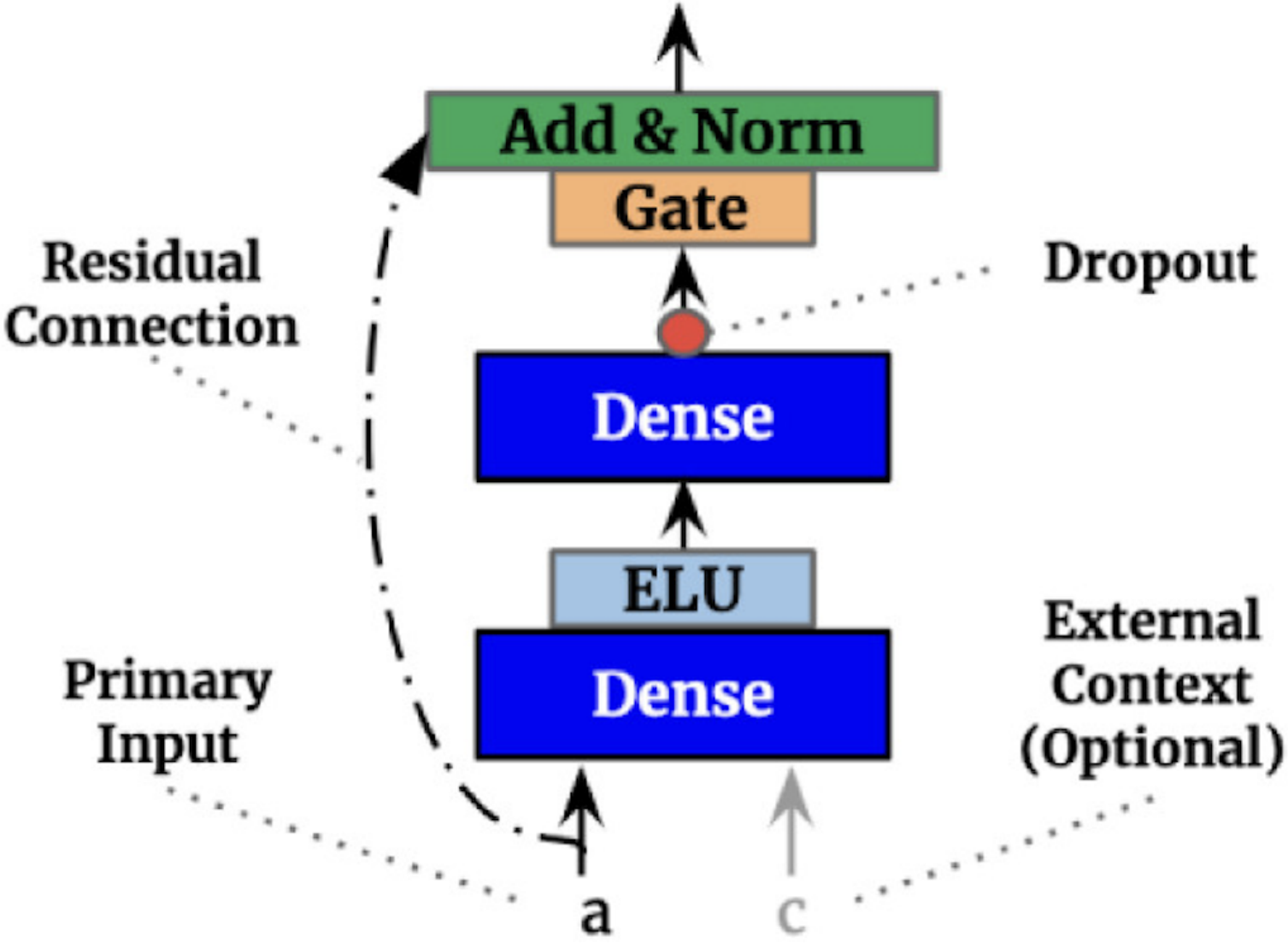

Figure: Gated Residual Network (GRN)

Gating with GRN

Exponential Linear Unit (ELU)

- Input is **a**, and context is **c**.
- ELU: $f(x) = \begin{cases} x, & x > 0 \\ \alpha\,(e^x - 1), & \textit{otherwise} \end{cases}$
- ELU($\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}$) such that when it's greater than '0' it's ($\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}$) (the output is the same as input). And when otherwise, it's a constant.

Gating with GRN

Gated Linear Unit (GLU)

- A sigmoid activation function $\sigma(\cdot)$ is '0' for values less than '0'.
- Hence $\sigma(\mathbf{W}\ \mathrm{ELU}(\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}) + b)$ is '0' for all values less than '0'.
- A $\mathrm{GLU}(\mathbf{x}) \ = \ \sigma(\mathbf{W}\ \mathbf{x} + \mathbf{b}) \odot (\mathbf{W}\ \mathbf{x} + \mathbf{b})$, where $\odot$ is an element-wise product operator.
- A $\mathrm{GLU}(\mathbf{W}\ \mathrm{ELU}(\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}) + b) \ = \ \sigma(\mathbf{W}\ \mathrm{ELU}(\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}) + b) \odot (\mathbf{W}\ \mathrm{ELU}(\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}) + b)$.
- A GLU doubles the dimensionality, initially, and passes the other half through a sigmoid activation function with values between '0' and '1', which are then multiplied with the original layer on an element-by-element basis reducing the dimension back.
- The resulting product is GLU. Basically, the gate modulates the linear projection $(\mathbf{W}\ \mathrm{ELU}(\mathbf{W}\ \mathbf{a} + \mathbf{W}\ \mathbf{c} + \mathbf{b}) + b)$.

Gating with GRN

Layer Normalization and Context

- When GLU output is nearly '0' implying without a context **c** and GRN is limited only to the original input **a**.
- Layer normalization helps with alleviating exploding and vanishing gradients due to updates to the input **a** at each time step resulting in a more stable hidden-to-hidden dynamics.
- Layer normalization re-centers and re-scales.
- $h_i = f(\frac{g_i}{\sigma_i}(a_i - u_i) + b_i)$, where u_i is the mean of input a_i, and σ_i is the standard deviation of the input.
- $\text{GRN}(\boldsymbol{a}, \boldsymbol{c}) = \text{LayerNorm}(\boldsymbol{a} + \text{GLU}(\boldsymbol{a}, \boldsymbol{c}))$.

Variable Selection Network Architecture

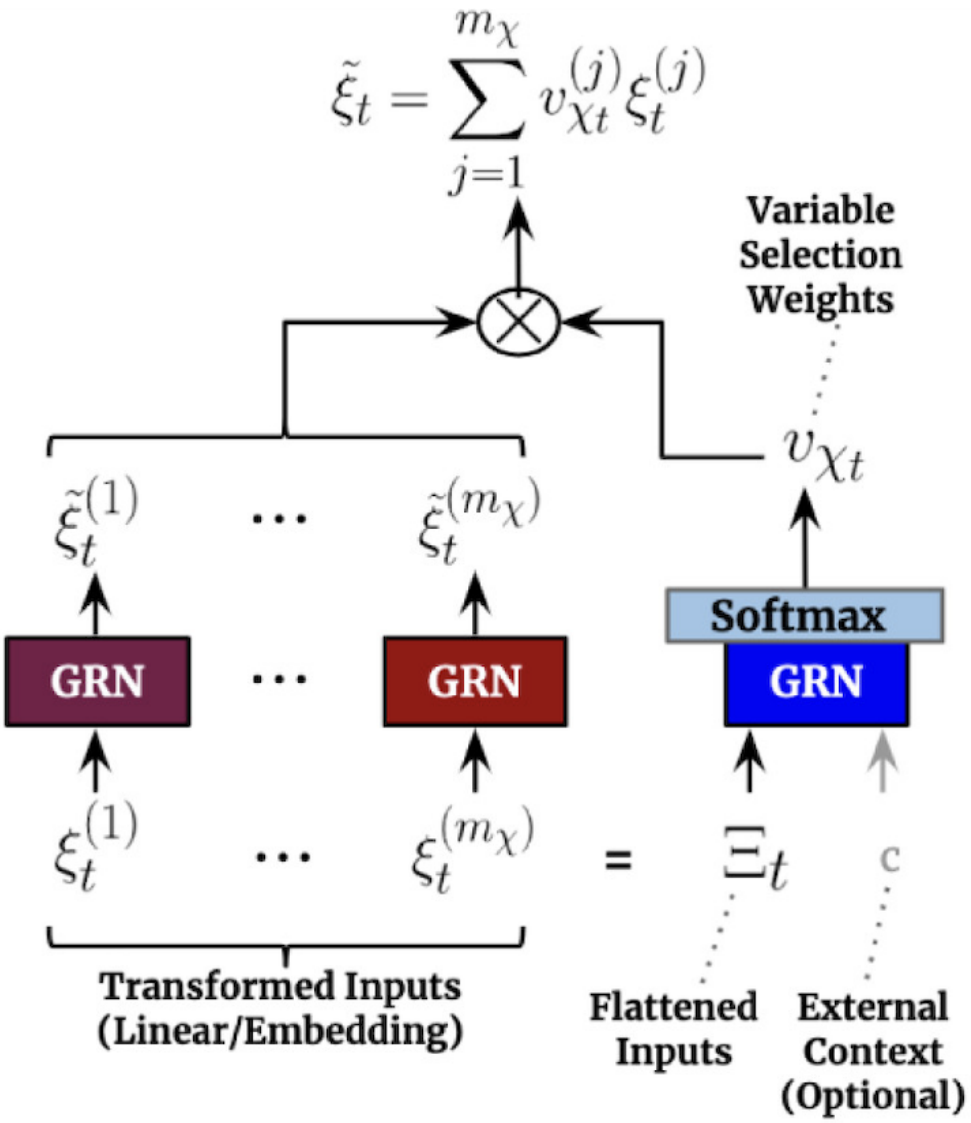

Figure: Variable Selection

Selecting Variables with Variables Selection Network

- Static features are projected in a high dimensional space through entity-embeddings, and continuous variables are vectorized using linear transformations.
- Separate variable selection networks for each static, past and future inputs.
- Varaible selection for the past or historical inputs are described here, for future and static inputs the method is similar.
- The method is described for a single time step t, and it is repeated at each time step.

Variables Selection Network

Past Inputs

- A vector of *j-th* past input at time step t is $\boldsymbol{\xi}_t^j$.
- $\boldsymbol{\Xi}_t = \left[\boldsymbol{\xi}_t^{1^T}, \boldsymbol{\xi}_t^{2^T}, \ldots, \boldsymbol{\xi}_t^{m_x{}^T}\right]$ is a flattened vector of all past inputs combined.
- A weight vector $\boldsymbol{\upsilon}_{\chi_t}$ is derived to weight each past input vector $\boldsymbol{\xi}_t^j$ by first applying GRN to the falttened input vector and the context vector $\mathbf{c}_s$.
- Then softmax is computed to find the weights on the output of GRN as follows $\boldsymbol{\upsilon}_{\chi_t} = \text{Softmax}(\text{GRN}(\boldsymbol{\Xi}_t, \mathbf{c}_s))$.
- A non-linear processing of each past input vector $\boldsymbol{\xi}_t^j$ creates a new vector $\tilde{\boldsymbol{\xi}}_t^j = \text{GRN}(\boldsymbol{\xi}_t^j)$.
- Finally, a processed past input vector is computed by combiming all past input vectors according to their weights $\tilde{\boldsymbol{\xi}}_t = \sum_{j=1}^{m_\chi} \boldsymbol{\upsilon}_{\chi_t}^j \tilde{\boldsymbol{\xi}}_t^j$.

Static Covariate Encoders

- A strength of TFT is to allow the use of static covariates with the time-varying inputs. To enable that statice covariate encoders create conext vectors.
- Four different GRNs are used to create four different conext vectors, viz., $\mathbf{c}_s, \mathbf{c}_e, \mathbf{c}_c$, and, $\mathbf{c}_h$, which are used in four different locations in the TFT.
- The four context includes:
  - $\mathbf{c}_s$ used in temporal variable selection.
  - $\mathbf{c}_c$ and $\mathbf{c}_h$ for local processing of temporal features.
  - $\mathbf{c}_e$ for enriching of temporal features with static information.
- Just like in the previous slide, let ζ be the output of static variable selection network. Then the static context vector for the temporal past variable selection network, $\mathbf{c}_s = \text{GRN}(\zeta)$.

Attention Network

- In TFT the Transformers based architrecture with the self-attention mechanism are used where the terms "query", "key", and "value" represent three distinct aspects of the input data that are used to compute attention scores and ultimately learn meaningful patterns and relationships within the data.
- In particular, the self-attention mechanisms allow to learn the long-term relationships across different time steps.
- The Query $\mathbf{Q}$ is a representation of a particular input element (e.g., a word or token in a sequence) that is used to determine how much attention it should pay to other elements in the sequence. In other words, the query is like a question posed by the model to identify which parts of the input are most relevant or informative for a given element.

Attention Network

- The Key $\mathbf{K}$ is a representation of every input element in the sequence, and it is used to match or compare with the query. The key can be thought of as a unique identifier for each input element that helps the model determine how relevant or related that element is to the query.
- The Value $\mathbf{V}$ is another representation of each input element, but its purpose is different from the key. Once the model has determined which elements in the sequence are most relevant to the query using the key, the value is used to compute a weighted sum, resulting in a new, context-aware representation of the input element that takes into account the relationships and dependencies within the sequence.
- The dot product of $\mathbf{Q}$ and $\mathbf{K}$ with Softmax determines the attention scores, $\mathrm{A}(\mathbf{Q}, \mathbf{K}) = \mathrm{Softmax}\left(\frac{\mathbf{Q}\ \mathbf{K}^T}{\sqrt{d^{attn}}}\right)$.
- The attention scores are used to weight the value $\mathbf{V}$ to give the attention layer output, $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = A(\mathbf{Q}, \mathbf{K})\ \mathbf{V}$.

Attention Network

Multi-Head Attention

- Multi-head attention extends the single attention network to learn multipile long-term relationships.
- It provides two advantages, first mulitple representaion subspaces one for each $\mathbf{Q}^h, \mathbf{K}, ^h\mathbf{V}^h$. Second, models ability to focus on different positions in time to learn multiple long-term relationships.
- Attention for a single head $\mathbf{H}_h$ =Attention($\mathbf{Q}^h, \mathbf{K}, ^h\mathbf{V}^h$).
- Interpretable Multi-Head($\mathbf{Q}, \mathbf{K}, \mathbf{V}$) $= \tilde{\mathbf{H}}\mathbf{W}_H$.
- Different heads for different representation subspaces, MultiHead($\mathbf{Q}, \mathbf{K}, \mathbf{V}$)$= [\mathbf{H}_1, \mathbf{H}_2, \ldots, \mathbf{H}_{m_H}]\mathbf{W}_H$.
- $\mathbf{W}_H$ linearly combines outputs from all heads $\mathbf{H}_h$.
- $\tilde{\mathbf{H}} = \tilde{\mathbf{A}}\ (\mathbf{Q}, \mathbf{K})\ \mathbf{V}$.
- $\tilde{\mathbf{H}} = \frac{1}{H}\sum_{h=1}^{m_H} \mathrm{A}(\mathbf{Q}^h, \mathbf{K}^h)\ \mathbf{V} = \frac{1}{H}\sum_{h=1}^{m_H} \mathrm{Attention}(\mathbf{Q}^h, \mathbf{K}^h, \mathbf{V})$.

Temporal Fusion Decoder

Locality (or Local Context)

- Learns one model per time series with methods such as ARIMA, Exponential Smoothing, and Bayesian time series. It captures individual random-effects for each time series. For example, whether an observed point is an anomaly, a change point, or part of a pattern due to events such as holidays, or extreme weather, etc.
- As mentioned above, some methods such as ARIMA, Gaussian Processes, etc. allow explicit modeling of the local shape of the curve (time series) around the observation point. These account for the random-effects at the observation as local flactuations.
- Self-attention networks based on RNNs and LSTMs lack the local context. Because these naturally account for long-term and global time series realtionships as discussed in multihead attention networks.
- Adding a convolutional self-attention through causal convolutions to produce Queries and Keys in the self-attention layer accounts for the local shape of the curve and random-effects around observation.

Temporal Fusion Decoder

Locality Enhancement with Sequence-to-Sequence Layer

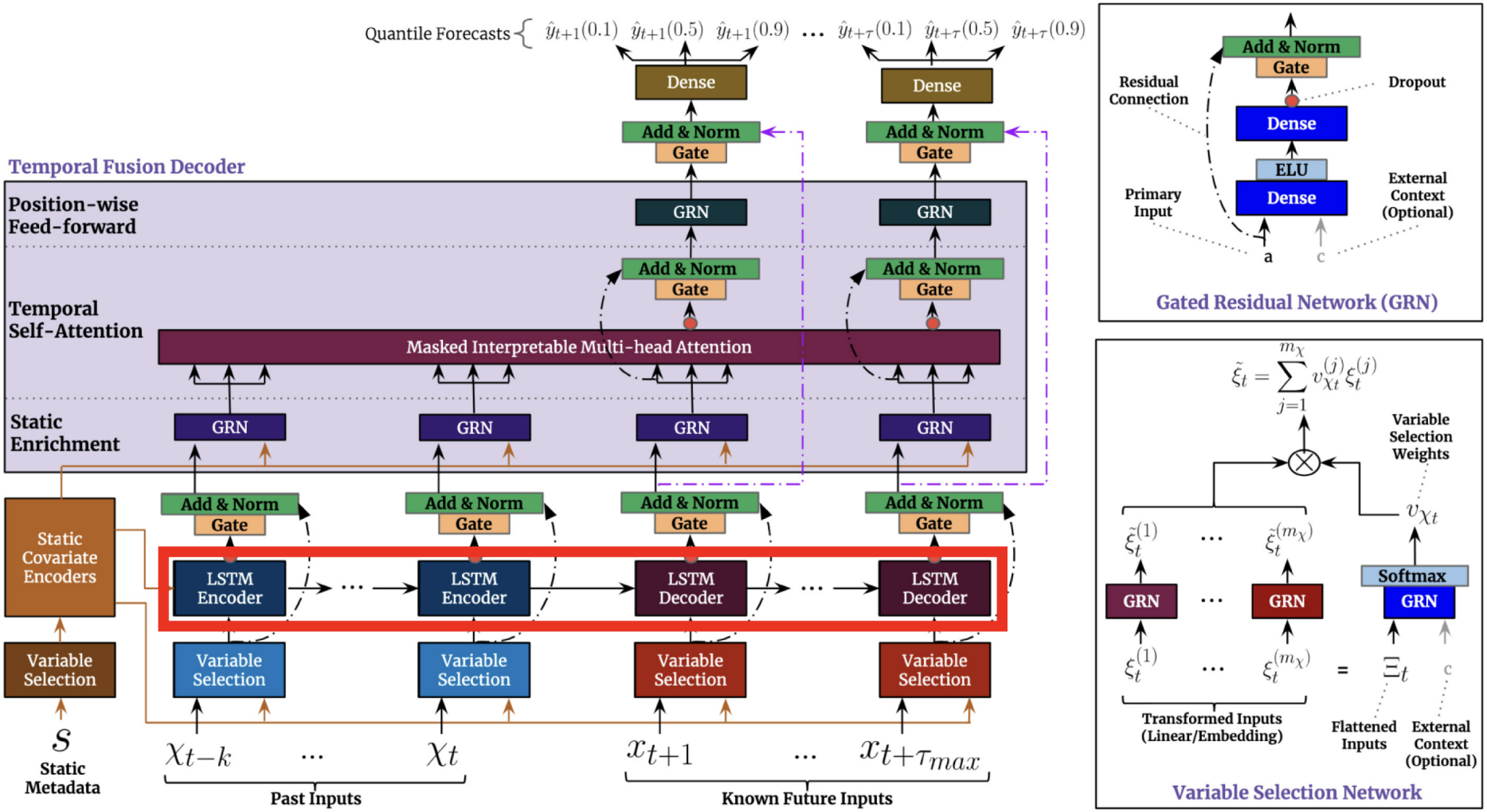

Figure: LSTM Encoder-Decoder Layer shown in the red frame.

Temporal Fusion Decoder

Locality Enhancement with Sequence-to-Sequence Layer

- A convoluational layer is not fit for TFT due to the presence of varying number of past and future inputs.
- Therefore, an LSTM encoder-decoder is added with the past inputs, $\tilde{\boldsymbol{\xi}}_{t-k:t}$ into the encoder, future inputs $\tilde{\boldsymbol{\xi}}_{t+1:t+\tau_{max}}$ into the decoder.
- This brings two benefits: First, it creates temporal features around the time-step t addressing the local context, and second, it does away with positional encoding an important part of the transformers.
- The temporal features from LSTM encoder-decoder, $\phi(t,n) \in \{\phi(t,-k),\ldots,\phi(t,\tau_{max})\}$ are inputs to the temporal fusion decoder, where n is the positional index.
- The gated connection is $\tilde{\phi}(t,n) =$LayerNorm($\tilde{\boldsymbol{\xi}}_{t+n}+GLU(\phi(t,n))$).

Temporal Fusion Decoder

Static Enrichment Layer

- In this layer, static variables are combined with the local temporal dynamics from sequence-to-sequence.
- $\theta(t, n) = \text{GRN}\left(\tilde{\phi}(t, n), \boldsymbol{c}_e\right)$.